

INDEX

[illegible]

Attendance Management System using Database Management System

Objective:

The Attendance Management System is designed to:

- Digitally record and manage student attendance
- Reduce manual errors and paperwork
- Provide quick retrieval of attendance reports
- Ensure data consistency using DBMS concepts

Scope of the Project:

- Manage students, faculty, subjects, and classes
- Record daily attendance
- Generate attendance reports
- Enforce integrity constraints using DBMS features

Database Design:

Entities

- Student
- Faculty
- Subject
- Course/Class
- Attendance

Student Table

```
mysql> CREATE TABLE Student (  
->     student_id INT PRIMARY KEY,  
->     name VARCHAR(50),  
->     roll_no VARCHAR(20) UNIQUE,  
->     course VARCHAR(50),  
->     semester INT  
-> );  
Query OK, 0 rows affected (0.13 sec)
```

Faculty Table

```
mysql> CREATE TABLE Faculty (  
->     faculty_id INT PRIMARY KEY,  
->     name VARCHAR(50),  
->     department VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.82 sec)
```

Subject Table

```
mysql> CREATE TABLE Subject (  
->     subject_id INT PRIMARY KEY,  
->     subject_name VARCHAR(50),  
->     faculty_id INT,  
->     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)  
-> );  
Query OK, 0 rows affected (3.50 sec)
```

Attendance Table

```
mysql> CREATE TABLE Attendance (  
->     attendance_id INT PRIMARY KEY,  
->     student_id INT,  
->     subject_id INT,  
->     date DATE,  
->     status ENUM('Present', 'Absent'),  
->     FOREIGN KEY (student_id) REFERENCES Student(student_id),  
->     FOREIGN KEY (subject_id) REFERENCES Subject(subject_id)  
-> );  
Query OK, 0 rows affected (0.24 sec)
```

Experiment – 2

Aim: Use DDL commands – CREATE, ALTER, DROP and TRUNCATE.

Create Command:

```
MariaDB [Attendance]> CREATE TABLE Email(  
    -> mail varchar(200)  
    -> );
```

```
MariaDB [Attendance]> SHOW TABLES;
```

```
+-----+  
| Tables_in_Attendance |  
+-----+  
| Attendance           |  
| Email                 |  
| Faculty               |  
| Student               |  
| Subject               |  
+-----+
```

ALTER Command:

ALTER command in SQL is used to modify the structure of an existing table in a database. It allows operations such as adding, deleting, or modifying columns and constraints. It does not affect the data stored in the table

Add :

```
MariaDB [Attendance]> ALTER TABLE Student  
    -> ADD Email varchar(200) ;
```

```
MariaDB [Attendance]> describe Student;
```

```
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| student_id | int(11)       | NO   | PRI | NULL    |       |  
| name       | varchar(50)   | YES  |     | NULL    |       |  
| roll_no    | varchar(20)   | YES  | UNI | NULL    |       |  
| course     | varchar(50)   | YES  |     | NULL    |       |  
| semester   | int(11)       | YES  |     | NULL    |       |  
| Email      | varchar(200)  | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+
```

Modify :

```
MariaDB [Attendance]> ALTER TABLE Student  
    -> MODIFY COLUMN Email INT;
```

```
MariaDB [Attendance]> describe Student;
```

Field	Type	Null	Key	Default	Extra
student_id	int(11)	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
roll_no	varchar(20)	YES	UNI	NULL	
course	varchar(50)	YES		NULL	
semester	int(11)	YES		NULL	
Email	int(11)	YES		NULL	

Drop :

```
MariaDB [Attendance]> DROP TABLE Email;
```

```
MariaDB [Attendance]> show tables;
```

Tables_in_Attendance
Attendance
Faculty
Student
Subject

TRUNCATE Command:

```
MariaDB [Attendance]> insert into Email values ('dev@gmail.com');
```

```
MariaDB [Attendance]> select * from Email;
```

mail
dev@gmail.com

```
MariaDB [Attendance]> TRUNCATE TABLE Email;
```

Query OK, 0 rows affected (0.036 sec)

```
MariaDB [Attendance]> select * from Email;
```

Empty set (0.001 sec)

Experiment – 3

Aim: Apply the integrity constraints like Primary key, Foreign Key, Check, NOT NULL, etc, to the tables.

PRIMARY KEY Constraint:

Here student_id is a primary key .

```
mysql> CREATE TABLE Student (  
->     student_id INT PRIMARY KEY,  
->     name VARCHAR(50),  
->     roll_no VARCHAR(20) UNIQUE,  
->     course VARCHAR(50),  
->     semester INT  
-> );  
Query OK, 0 rows affected (0.13 sec)
```

FOREIGN KEY Constraint:

```
mysql> CREATE TABLE Attendance (  
->     attendance_id INT PRIMARY KEY,  
->     student_id INT,  
->     subject_id INT,  
->     date DATE,  
->     status ENUM('Present', 'Absent'),  
->     FOREIGN KEY (student_id) REFERENCES Student(student_id),  
->     FOREIGN KEY (subject_id) REFERENCES Subject(subject_id)  
-> );  
Query OK, 0 rows affected (0.24 sec)
```

```
mysql> DESCRIBE Attendance;
```

Field	Type	Null	Key	Default	Extra
attendance_id	int	NO	PRI	NULL	
student_id	int	YES	MUL	NULL	
subject_id	int	YES	MUL	NULL	
date	date	YES		NULL	
status	enum('Present', 'Absent')	YES		NULL	

```
5 rows in set (0.00 sec)
```

CHECK Constraint:

```
mysql> ALTER TABLE STUDENT ADD CHECK (semester BETWEEN 1 AND 8);
Query OK, 0 rows affected (0.17 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO STUDENT VALUES(101,'ATHARVA','81','BTECH',9);
ERROR 3819 (HY000): Check constraint 'student_chk_1' is violated.
```

NOT NULL Constraint:

```
mysql> ALTER TABLE Student MODIFY name VARCHAR(50) NOT NULL;
Query OK, 0 rows affected (0.15 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> DESCRIBE Student;
```

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	
name	varchar(50)	NO		NULL	
roll_no	varchar(20)	YES	UNI	NULL	
course	varchar(50)	YES		NULL	
semester	int	YES		NULL	

5 rows in set (0.00 sec)

UNIQUE Constraint:

```
mysql> ALTER TABLE Student MODIFY roll_no VARCHAR(50) NOT NULL UNIQUE;
Query OK, 0 rows affected, 1 warning (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 1
```

```
mysql> DESCRIBE Student;
```

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	
name	varchar(50)	NO		NULL	
roll_no	varchar(50)	NO	UNI	NULL	
course	varchar(50)	YES		NULL	
semester	int	YES		NULL	

5 rows in set (0.00 sec)

Experiment – 4

Aim: Use basic DML commands- SELECT, INSERT, UPDATE and DELETE.

INSERT Command:

```
mysql> INSERT INTO STUDENT VALUES(101,'ATHARVA','81','BTECH',4);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO STUDENT VALUES(102,'NICK','80','BTECH',4);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> INSERT INTO STUDENT VALUES(103,'RISHABH','88','BTECH',5);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO STUDENT VALUES(104,'YASH','89','BBA',6);  
Query OK, 1 row affected (0.01 sec)
```

SELECT Command:

```
mysql> SELECT * FROM Student;
```

student_id	name	roll_no	course	semester
101	ATHARVA	81	BTECH	4
102	NICK	80	BTECH	4
103	RISHABH	88	BTECH	5
104	YASH	89	BBA	6

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM Student WHERE course = "BTECH";
```

student_id	name	roll_no	course	semester
101	ATHARVA	81	BTECH	4
102	NICK	80	BTECH	4
103	RISHABH	88	BTECH	5

3 rows in set (0.00 sec)

```
mysql> SELECT student_id,name,course FROM Student WHERE semester = 4;
```

student_id	name	course
101	ATHARVA	BTECH
102	NICK	BTECH

2 rows in set (0.00 sec)

UPDATE Command:

```
mysql> UPDATE Student SET semester = 5 where course = 'BTECH';  
Query OK, 2 rows affected (0.01 sec)  
Rows matched: 3  Changed: 2  Warnings: 0
```

```
mysql> SELECT * FROM Student;
```

student_id	name	roll_no	course	semester
101	ATHARVA	81	BTECH	5
102	NICK	80	BTECH	5
103	RISHABH	88	BTECH	5
104	YASH	89	BBA	6

4 rows in set (0.00 sec)

DELETE Command:

```
mysql> DELETE FROM STUDENT WHERE course != 'BTECH';  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM Student;
```

student_id	name	roll_no	course	semester
101	ATHARVA	81	BTECH	5
102	NICK	80	BTECH	5
103	RISHABH	88	BTECH	5

3 rows in set (0.00 sec)

Experiment – 5

Aim: Writing queries for implementing Built-in functions, GROUP BY, HAVING and ORDER BY.

GROUP BY:

Used to **group rows having the same values** in one or more columns so that aggregate functions (like COUNT, SUM, AVG) can be applied to each group.

HAVING:

Used to **filter grouped data** after applying GROUP BY. It works with aggregate functions.

ORDER BY:

Used to **sort the result set** in ascending (ASC) or descending (DESC) order based on one or more columns.

Q: Display all students from semester 3 using WHERE clause.

```
mysql> SELECT * FROM student WHERE semester = 3;
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
5	Arjun	105	BTech	3

Q: Display student records sorted by name using ORDER BY.

```
mysql> SELECT * FROM student ORDER BY name;
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
5	Arjun	105	BTech	3
6	Karan	106	BTech	4
3	Rohit	103	BTech	2
4	Sneha	104	BTech	4

Q: Display total attendance records using COUNT().

```
mysql> SELECT COUNT(*) as total FROM attendance ;
```

total
41

Q: Display number of classes conducted per subject using GROUP BY.

```
mysql> SELECT subject_id , COUNT(*) FROM attendance GROUP BY subject_id;
```

subject_id	COUNT(*)
101	13
102	11
103	9
104	7
105	1

Q: Display students having attendance percentage less than 75% using HAVING.

```
mysql> SELECT student_id, SUM(CASE status WHEN "Present" THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS Total_Attendance FROM attendance GROUP BY student_id HAVING Total_Attendance < 75;
```

student_id	Total_Attendance
2	57.14286
3	40.00000
5	20.00000
6	66.66667

Q: Display the distinct courses offered.

```
mysql> SELECT DISTINCT subject_id, subject_name FROM subject;
```

subject_id	subject_name
101	DBMS
102	Operating System
103	Computer Networks
104	Python
105	Java

Q: Display students whose name starts with „A“ using LIKE.

```
mysql> SELECT * FROM student WHERE NAME LIKE "A%";
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
5	Arjun	105	BTech	3

Q: Display students between semester 2 and 4 using BETWEEN.

```
mysql> SELECT * FROM student WHERE semester BETWEEN 2 AND 4;
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
3	Rohit	103	BTech	2
4	Sneha	104	BTech	4
5	Arjun	105	BTech	3
6	Karan	106	BTech	4

Q: Display students in semester 3 or 4 using IN.

```
mysql> SELECT * FROM student WHERE semester IN(3,4);
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
4	Sneha	104	BTech	4
5	Arjun	105	BTech	3
6	Karan	106	BTech	4

Q: Display all students ordered by name in ascending order.

```
mysql> SELECT * FROM student ORDER BY name;
```

student_id	name	roll_no	course	semester
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
5	Arjun	105	BTech	3
6	Karan	106	BTech	4
3	Rohit	103	BTech	2
4	Sneha	104	BTech	4

Q: Display the list of all students sorted by semester in descending order.

```
mysql> SELECT * FROM student ORDER BY semester DESC;
```

student_id	name	roll_no	course	semester
4	Sneha	104	BTech	4
6	Karan	106	BTech	4
1	Aman	101	BTech	3
2	Anjali	102	BTech	3
5	Arjun	105	BTech	3
3	Rohit	103	BTech	2

Q: Display attendance records sorted by date.

```
mysql> SELECT * FROM attendance ORDER BY date;
```

attendance_id	student_id	subject_id	date	status
1	1	101	2026-02-01	Present
30	5	101	2026-02-01	Absent
24	4	101	2026-02-01	Present
19	3	101	2026-02-01	Absent
35	6	101	2026-02-01	Present
12	2	101	2026-02-01	Present
2	1	102	2026-02-02	Present
13	2	102	2026-02-02	Absent
20	3	102	2026-02-02	Absent
31	5	102	2026-02-02	Absent
25	4	102	2026-02-02	Present
36	6	102	2026-02-02	Present
3	1	103	2026-02-03	Present
37	6	103	2026-02-03	Absent
14	2	103	2026-02-03	Present
21	3	103	2026-02-03	Present
26	4	103	2026-02-03	Present
32	5	103	2026-02-03	Absent
4	1	104	2026-02-04	Present
22	3	104	2026-02-04	Absent
27	4	104	2026-02-04	Present
15	2	104	2026-02-04	Absent
33	5	104	2026-02-04	Present
38	6	104	2026-02-04	Present
5	1	101	2026-02-05	Present
28	4	101	2026-02-05	Present
16	2	101	2026-02-05	Present
23	3	101	2026-02-05	Present

34	5	101	2026-02-05	Absent
39	6	101	2026-02-05	Present
6	1	102	2026-02-06	Present
17	2	102	2026-02-06	Absent
29	4	102	2026-02-06	Present
40	6	102	2026-02-06	Absent
7	1	103	2026-02-07	Absent
18	2	103	2026-02-07	Present
8	1	104	2026-02-08	Present
9	1	101	2026-02-09	Present
10	1	102	2026-02-10	Present
11	1	103	2026-02-11	Present
41	1	105	2026-02-12	Present

Experiment – 6

Aim: Writing queries for implementing joins.

INNER JOIN:

Returns only the rows where there is a **match in both tables**.

LEFT JOIN (LEFT OUTER JOIN):

Returns **all rows from left table** and matching rows from right table; non-matching rows return NULL.

RIGHT JOIN (RIGHT OUTER JOIN):

Returns **all rows from right table** and matching rows from left table; non-matching rows return NULL.

FULL JOIN:

Returns **all rows from both tables**, with NULLs where there is no match.

NATURAL JOIN:

Automatically joins two tables based on **columns with the same name and compatible data types**, removing duplicate columns in the result.

Q: Display student name, subject name, date and status using JOIN.

```
mysql> SELECT name, subject_name, date, status from student NATURAL JOIN attendance NATURAL JOIN subject;
```

name	subject_name	date	status
Aman	DBMS	2026-02-01	Present
Aman	Operating System	2026-02-02	Present
Aman	Computer Networks	2026-02-03	Present
Aman	Python	2026-02-04	Present
Aman	DBMS	2026-02-05	Present
Aman	Operating System	2026-02-06	Present
Aman	Computer Networks	2026-02-07	Absent
Aman	Python	2026-02-08	Present
Aman	DBMS	2026-02-09	Present
Aman	Operating System	2026-02-10	Present
Aman	Computer Networks	2026-02-11	Present
Aman	Java	2026-02-12	Present
Anjali	DBMS	2026-02-01	Present
Anjali	Operating System	2026-02-02	Absent
Anjali	Computer Networks	2026-02-03	Present
Anjali	Python	2026-02-04	Absent
Anjali	DBMS	2026-02-05	Present
Anjali	Operating System	2026-02-06	Absent
Anjali	Computer Networks	2026-02-07	Present
Rohit	DBMS	2026-02-01	Absent
Rohit	Operating System	2026-02-02	Absent
Rohit	Computer Networks	2026-02-03	Present
Rohit	Python	2026-02-04	Absent
Rohit	DBMS	2026-02-05	Present
Sneha	DBMS	2026-02-01	Present
Sneha	Operating System	2026-02-02	Present
Sneha	Computer Networks	2026-02-03	Present
Sneha	Python	2026-02-04	Present
Sneha	DBMS	2026-02-05	Present

Sneha	Operating System	2026-02-06	Present
Arjun	DBMS	2026-02-01	Absent
Arjun	Operating System	2026-02-02	Absent
Arjun	Computer Networks	2026-02-03	Absent
Arjun	Python	2026-02-04	Present
Arjun	DBMS	2026-02-05	Absent
Karan	DBMS	2026-02-01	Present
Karan	Operating System	2026-02-02	Present
Karan	Computer Networks	2026-02-03	Absent
Karan	Python	2026-02-04	Present
Karan	DBMS	2026-02-05	Present
Karan	Operating System	2026-02-06	Absent

Q: Calculate attendance percentage for each student

```
mysql> SELECT name, SUM(CASE status WHEN "Present" THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS Attendance_Percentage FROM student NATURAL JOIN attendance GROUP BY name;
```

name	Attendance_Percentage
Aman	91.66667
Anjali	57.14286
Rohit	40.00000
Sneha	100.00000
Arjun	20.00000
Karan	66.66667

Q: List students absent on a specific date.

```
mysql> SELECT student_id, name FROM attendance NATURAL JOIN student WHERE status = "Absent" AND DATE = "2026-02-04";
```

student_id	name
2	Anjali
3	Rohit

Q: Display subject-wise present count

```
mysql> SELECT subject_name, subject_id, COUNT(*) FROM subject NATURAL JOIN attendance WHERE status = "Present" GROUP BY subject_id;
```

subject_name	subject_id	COUNT(*)
DBMS	101	10
Operating System	102	6
Computer Networks	103	6
Python	104	5
Java	105	1

Q: Find the student with maximum attendance percentage.

```
mysql> SELECT student_id, name, SUM(CASE status WHEN "Present" THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS Attendance_Percentage
-> FROM student NATURAL JOIN attendance
-> GROUP BY student_id
-> ORDER BY
-> Attendance_Percentage DESC
-> LIMIT 1;
```

student_id	name	Attendance_Percentage
4	Sneha	100.00000

Q: Display students having attendance percentage greater than 80

```
mysql> SELECT name, SUM(CASE status WHEN "Present" THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS Attendance_Percentage FROM student NATURAL JOIN attendance GROUP BY name HAVING Attendance_Percentage > 80 ;
```

name	Attendance_Percentage
Aman	91.66667
Sneha	100.00000

Q: Display students who were present more than 10 times.

```
mysql> SELECT student_id, name FROM attendance NATURAL JOIN student GROUP BY student_id HAVING SUM(CASE status WHEN -> "Present" THEN 1 ELSE 0 END) > 10;
```

student_id	name
1	Aman

Q: Display subjects having more than 5 attendance records.

```
mysql> SELECT subject_id, subject_name FROM subject NATURAL JOIN attendance GROUP BY subject_id HAVING COUNT(*) > 5;
```

subject_id	subject_name
101	DBMS
102	Operating System
103	Computer Networks
104	Python

Q: Display students whose total attendance records are greater than 8

```
mysql> SELECT student_id ,name FROM student NATURAL JOIN attendance GROUP BY student_id HAVING COUNT(*) > 8;
```

student_id	name
1	Aman

Q: Display subjects where absent count is greater than 3

```
mysql> SELECT subject_id, subject_name FROM subject NATURAL JOIN attendance GROUP BY subject_id HAVING SUM(CASE status WHEN "Absent" THEN 1 ELSE 0 END) > 3;
```

subject_id	subject_name
102	Operating System

Experiment – 6

Aim: Writing queries for implementing sub queries.

Q: Write a query using a sub query to find students having attendance above average.

```
mysql> SELECT student_id, name, COUNT(*) AS total_classes FROM student NATURAL JOIN attendance
-> GROUP BY student_id
-> HAVING total_classes > (
-> SELECT AVG(total_count) FROM (
-> SELECT COUNT(*) AS total_count FROM attendance GROUP BY student_id) as avg_table );
```

student_id	name	total_classes
1	Aman	12
2	Anjali	7

Q: Display subjects where present count is greater than absent count

```
mysql> SELECT subject_id, subject_name FROM subject NATURAL JOIN attendance GROUP BY subject_id HAVING SUM(CASE status WHEN "Present" THEN 1 ELSE 0 END) > SUM(CASE status WHEN "Absent" THEN 1 ELSE 0 END) ;
```

subject_id	subject_name
101	DBMS
102	Operating System
103	Computer Networks
104	Python
105	Java